

Manual Técnico do Script `create_rt_ticket_abuse.py` - Sistema Hórus

Versão do Documento: 1.0

Data de Atualização: Dezembro de 2025

Tecnologia Principal: Python 3 + PostgreSQL + cURL (via subprocess)

Este manual técnico descreve em detalhes o script `create_rt_ticket_abuse.py`, que atua como o módulo de automação de alertas para credenciais vazadas oriundas de Abuse/Zimbra no sistema Hórus. O script consulta o banco de dados por registros inseridos no dia atual (excluindo fontes BTTNG), gera relatórios mascarados, cria payloads para tíquetes e envia para o Request Tracker (RT) via cURL com autenticação token e certificado. Ele garante notificação automática da equipe de segurança sobre novos vazamentos.

O documento explica as funcionalidades principais, configurações, funções disponíveis, restrições de uso e mecanismos de integração implementados. Ele é destinado a administradores, desenvolvedores e mantenedores que precisam compreender ou modificar o comportamento da criação de tíquetes para fontes Abuse/Zimbra.

1. Introdução e Propósito

O `create_rt_ticket_abuse.py` é o componente dedicado à criação automática de tíquetes no RT para credenciais vazadas detectadas via Abuse/Zimbra no Hórus, uma plataforma de Threat Intelligence. Ele filtra registros do dia atual (`source_bttng` falso ou nulo), mascara senhas para privacidade, gera arquivos de anexo (tabela formatada) e payload do tíquete, e submete via cURL. O script opera de forma autônoma via execução direta ou integrado a fluxos (ex: pós-processamento em `zimbra_parser.py`), com logging básico para monitoramento.

Ele utiliza `psycopg2` para consultas ao PostgreSQL, `subprocess` para cURL e `datetime` para timestamps. O foco é em alertas seguros, com remoção opcional de arquivos temporários após envio bem-sucedido.

1.1 Dependências e Integrações

- **Linguagem:** Python 3.
- **Banco de Dados:** PostgreSQL (via psycopg2), com conexões simples.
- **Bibliotecas:** os e sys (paths), subprocess (cURL), logging (logs básicos), datetime (datas).
- **Integrações Externas:** RT via API REST (URL e token), certificado PEM local para SSL.
- **Scripts Relacionados:** Chamado por zimbra_parser.py (pós-extração), config.py (configurações DB), credenciais_processor.py (dados processados).

2. Funcionalidades Principais

O `create_rt_ticket_abuse.py` fornece automação para alertas de vazamentos Abuse/Zimbra, com foco em consulta, formatação e submissão. Abaixo, as principais funcionalidades:

- **Consulta ao Banco:** Busca credenciais do dia atual (`DATE(data_insercao) = CURRENT_DATE`, `source_bttnng` falso/nulo), ordenadas por inserção DESC.
- **Mascaramento de Senhas:** Oculta senhas (primeiro/último visíveis, asteriscos no meio) para anexos seguros.
- **Geração de Arquivos:** Cria tabela TXT formatada (usuário/site/senha mascarada) como anexo e payload TXT para tíquete (com metadados como total de registros).
- **Submissão via cURL:** Envia multipart (content e attachment) para RT com cacert, token e headers.
- **Logging e Encerramento:** Logs info/error em cada etapa; encerra se sem resultados ou erros.
- **Remoção Temporários:** Opcional: Remove arquivos após envio bem-sucedido (try/except para evitar falhas).

3. Configurações

O script define configurações globais e de paths via os/sys:

- **Logging:** BasicConfig level=INFO, format com timestamp/name/level/message.
- **Query SQL:** SQL_QUERY_TODAY fixa (usuário/site/senha, filtro data/source_bttnng).
- **Paths:** CURRENT_DIR (dirname(file)), CERT_PATH (rt.pem relativo), arquivos gerados no dir atual (cred_abuse_[timestamp].txt, rt_abuse_[timestamp].txt).
- **RT:** RT_URL fixa, RT_TOKEN hardcoded (sensível – considere .env).
- **Formatação:** COL1/2/3_WIDTH (43/38/30) para colunas usuário/site/senha.
- **Banco de Dados:** Usa get_db_config_by_environment()['repo'] para conn.

Restrições: Token hardcoded (risco segurança); paths relativos (assume execução em /Horus/rt); logging sem file handler (stdout apenas).

4. Funções e Métodos Principais

O script define funções modulares para cada etapa. Cada função é descrita abaixo, incluindo parâmetros, funcionalidades e restrições:

- **fetch_todays_credentials():** Conecta ao DB, executa SQL_QUERY_TODAY, fetchall. Loga count. Retorna lista de tuples ou None em erros.
- **mask_password(password):** Mascara str (first + ''(len-2) + last se >2 chars). Retorna str.
- **generate_files(results):** Gera timestamp, cria header/separator/lines para tabela (ljust widths), escreve cred_filename. Cria content para ticket_filename com metadados. Retorna files ou None em IOErrors.
- **submit_ticket(ticket_file, attachment_file):** Verifica cert, constrói cmd cURL (POST, cacert, Authorization token, -F content/attachment). Run subprocess, verifica stdout por "200 Ok"/"Ticket". Remove files se sucesso (opcional). Sem retorno.
- **main():** Chama fetch (encerra se vazio/None), generate (encerra se None), submit.
- **if name == "main":** Chama main().

5. Restrições e Segurança

- **Filtros:** Apenas registros do dia atual e source_bttnng falso (Abuse/Zimbra); sem suporte a períodos custom.
- **Segurança:** Token hardcoded (mova para .env); cacert para SSL; máscaras em senhas. Subprocess check=True (raise em erros cURL).
- **Performance:** Fetchall simples (OK para volumes diários); widths fixas (corta strings longas).
- **Erros:** Logs error em exceções; encerra em falhas críticas (ex: import, DB conn). Sem retry em cURL.
- **Restrições Gerais:** Depende de cURL instalado; RT config fixa (Queue: General, Priority:0); arquivos no dir atual (risco overwrite). Sem auditoria DB (apenas logs).

Nota Final

O `create_rt_ticket_abuse.py` é simples e eficaz para alertas Abuse/Zimbra, mas alterações devem preservar máscaras e logging para conformidade (ex: LGPD para senhas). Consulte o código para extensões ou integre com email fallback. Em produção, mova token para segredos e monitore logs para falhas de submissão.

Manual Técnico do Script `create_rt_ticket_daily.py` - Sistema Hórus

Versão do Documento: 1.0

Data de Atualização: Dezembro de 2025

Tecnologia Principal: Python 3 + PostgreSQL + cURL (via subprocess)

Este manual técnico descreve em detalhes o script `create_rt_ticket_daily.py`, que atua como o módulo de automação de alertas para credenciais vazadas oriundas de BTTNG no sistema Hórus. O script consulta o banco de dados por registros processados no dia atual (source_bttnng 't'), gera relatórios mascarados, cria payloads para tíquetes e envia para o Request Tracker (RT) via cURL com autenticação token e certificado. Ele garante notificação automática da equipe de segurança sobre novos vazamentos diários.

O documento explica as funcionalidades principais, configurações, funções disponíveis, restrições de uso e mecanismos de integração implementados. Ele é destinado a administradores, desenvolvedores e mantenedores que precisam compreender ou modificar o comportamento da criação de tíquetes para fontes BTTNG.

1. Introdução e Propósito

O `create_rt_ticket_daily.py` é o componente dedicado à criação automática de tíquetes no RT para credenciais vazadas detectadas via BTTNG no Hórus, uma plataforma de Threat Intelligence. Ele filtra registros do dia atual (`DATE(processed_at) = CURRENT_DATE, source_bttng 't'`), mascara senhas para privacidade, gera arquivos de anexo (tabela formatada) e payload do tíquete, e submete via cURL. O script opera de forma autônoma via execução direta ou integrado a fluxos (ex: pós-sync em `bttng_integration.py`), com logging básico para monitoramento.

Ele utiliza `psycopg2` para consultas ao PostgreSQL, `subprocess` para cURL e `datetime` para timestamps. O foco é em alertas seguros, sem remoção automática de arquivos temporários.

1.1 Dependências e Integrações

- **Linguagem:** Python 3.
- **Banco de Dados:** PostgreSQL (via `psycopg2`), com conexões simples.
- **Bibliotecas:** `os` e `sys` (paths), `subprocess` (cURL), `logging` (logs básicos), `datetime` (datas).
- **Integrações Externas:** RT via API REST (URL e token), certificado PEM absoluto para SSL.
- **Scripts Relacionados:** Chamado por `bttng_integration.py` (pós-sync), `config.py` (configurações DB), `credenciais_processor.py` (dados processados).

2. Funcionalidades Principais

O `create_rt_ticket_daily.py` fornece automação para alertas de vazamentos BTTNG diários, com foco em consulta, formatação e submissão. Abaixo, as principais funcionalidades:

- **Consulta ao Banco:** Busca credenciais do dia atual (`DATE(processed_at) = CURRENT_DATE, source_bttng 't'`), ordenadas por `processed_at` DESC.
- **Mascaramento de Senhas:** Oculta senhas (primeiro/último visíveis, asteriscos no meio) para anexos seguros.
- **Geração de Arquivos:** Cria tabela TXT formatada (usuário/site/senha mascarada) como anexo e payload TXT para tíquete (com metadados como total de registros).
- **Submissão via cURL:** Envia multipart (content e attachment) para RT com cacert, token e headers.
- **Logging e Encerramento:** Logs info/error em cada etapa; encerra se sem resultados ou erros.

3. Configurações

O script define configurações globais e de paths via `os/sys`:

- **Logging:** BasicConfig level=INFO, format com timestamp/name/level/message.
- **Query SQL:** `SQL_QUERY_TODAY` fixa (usuário/site/senha, filtro `processed_at/source_bttng`).
- **Paths:** `CERT_PATH` absoluto (`/home/spf/GIT/Horus/rt/rt.pem`), arquivos gerados no dir atual (`cred_[date].txt`, `rt_[date].txt`).
- **RT:** `RT_URL` fixa, `RT_TOKEN` hardcoded (sensível – considere `.env`).
- **Formatação:** `COL1/2/3_WIDTH` (43/38/30) para colunas usuário/site/senha.
- **Banco de Dados:** Usa `get_db_config_by_environment()['repo']` para conn.

Restrições: Token hardcoded (risco segurança); cert path absoluto (depende de usuário `spf`); logging sem file handler (stdout apenas).

4. Funções e Métodos Principais

O script define funções modulares para cada etapa. Cada função é descrita abaixo, incluindo parâmetros, funcionalidades e restrições:

- **fetch_todays_credentials()**: Conecta ao DB, executa SQL_QUERY_TODAY, fetchall. Loga count. Retorna lista de tuples ou None em erros.
- **mask_password(password)**: Mascara str (first + ''(len-2) + last se >2 chars). Retorna str.
- **generate_files(results)**: Gera date str, cria header/separator/lines para tabela (ljust widths), escreve cred_filename. Cria content para ticket_filename com metadados. Retorna files ou None em IOErrors.
- **submit_ticket(ticket_file, attachment_file)**: Verifica cert, constrói cmd cURL (POST, cacert, Authorization token, -F content/attachment). Run subprocess, verifica stdout por "200 Ok"/"Ticket". Loga resposta completa.
- **main()**: Chama fetch (encerra se vazio/None), generate (encerra se None), submit.
- **if name == "main"**: Chama main().

5. Restrições e Segurança

- **Filtros**: Apenas registros do dia atual e source_bttnng 't' (BTTNG); sem suporte a períodos custom.
- **Segurança**: Token hardcoded (mova para .env); cacert para SSL; máscaras em senhas. Subprocess check=True (raise em erros cURL).
- **Performance**: Fetchall simples (OK para volumes diários); widths fixas (corta strings longas).
- **Erros**: Logs error em exceções; encerra em falhas críticas (ex: import, DB conn). Sem retry em cURL.
- **Restrições Gerais**: Depende de cURL instalado; RT config fixa (Queue: General, Priority:0); arquivos no dir atual (risco overwrite). Sem remoção de temporários (mantenha para debug).

Nota Final

O `create_rt_ticket_daily.py` é simples e eficaz para alertas BTTNG diários, mas alterações devem preservar máscaras e logging para conformidade (ex: LGPD para senhas). Consulte o código para extensões ou integre com email fallback. Em produção, mova token para segredos e monitore logs para falhas de submissão.